

Tentu, mari kita bedah kode ImageEditorService.cs ini.

Secara garis besar, kelas ini berfungsi sebagai "**Service Layer**" atau jembatan penghubung (Facade) antara antarmuka pengguna (UI) dan logika pemrosesan gambar yang lebih rumit (yang ada di folder Logic). Kelas ini menyimpan *state* (keadaan) gambar saat ini dan menyediakan metode-metode yang mudah dipanggil untuk memanipulasi gambar tersebut.

Berikut adalah penjelasan detail per bagian dan per baris kodenya:

1. Bagian Import (Using Statements)

C#

```
using System.Drawing;  
using System.Windows.Forms;  
using MiniPhotoshop.Logic.Helpers;  
using MiniPhotoshop.Logic.ImageProcessing;  
using MiniPhotoshop.Logic.Histogram;
```

- System.Drawing: Mengimpor pustaka standar C# untuk grafis, memungkinkan penggunaan kelas Bitmap, Color, dan Point.
- System.Windows.Forms: Digunakan kemungkinan untuk tipe data umum UI, meskipun di sini utamanya untuk interaksi standar.
- MiniPhotoshop.Logic...: Mengimpor *namespace* buatan sendiri yang berisi logika inti (seperti ImageDataProcessor, ImageNegation, dll) agar kelas ini bisa memanggil fungsi-fungsi tersebut.

2. Definisi Kelas dan Properti

C#

```
namespace MiniPhotoshop.Logic  
{  
    public class ImageEditorService  
    {
```

```
public Bitmap OriginalImage { get; private set; }  
private int[,,_] _imageData3D;
```

```
public bool IsImageLoaded => OriginalImage != null;
```

- **public class ImageEditorService:** Mendefinisikan kelas publik yang bisa diakses dari mana saja dalam proyek.
- **public Bitmap OriginalImage { get; private set; }:**
 - Menyimpan gambar **asli** yang sedang diedit.
 - **get:** Bisa dibaca oleh kelas lain.
 - **private set:** Hanya bisa diubah/diisi oleh kelas ini sendiri (untuk keamanan data).
- **private int[,,_] _imageData3D;:**
 - Menyimpan representasi data piksel gambar dalam bentuk array 3 dimensi (kemungkinan formatnya [lebar, tinggi, kanal_warna]).
 - Ini biasanya digunakan untuk pemrosesan gambar yang lebih cepat dibandingkan memanipulasi Bitmap secara langsung.
- **public bool IsImageLoaded => OriginalImage != null;:**
 - Properti *shortcut* (expression-bodied member). Mengembalikan true jika OriginalImage sudah ada isinya, dan false jika masih kosong (null).

3. Inisialisasi Gambar

C#

```
public void InitializeImage(Bitmap image)  
{  
    OriginalImage = new Bitmap(image);  
    _imageData3D = ImageDataProcessor.LoadTo3DArray(OriginalImage);  
}
```

- **OriginalImage = new Bitmap(image);:** Membuat salinan baru dari gambar yang diinput. Ini penting agar jika gambar sumber di luar kelas ini berubah, gambar di dalam Service ini tidak ikut rusak (memutuskan referensi).
- **_imageData3D = ImageDataProcessor...:** Memanggil kelas bantuan ImageDataProcessor untuk mengonversi gambar Bitmap menjadi array 3D agar siap diproses lebih lanjut.

4. Manajemen State Gambar

C#

```
public Bitmap GetRestoredImage()
{
    if (!IsImageLoaded) return null;
    return (Bitmap)OriginalImage.Clone();
}
```

- **if (!IsImageLoaded) return null;**: Pengecekan keamanan (Guard Clause). Jika belum ada gambar, hentikan proses dan kembalikan null.
- **return (Bitmap)OriginalImage.Clone();**: Mengembalikan gambar asli. **Penting:** Menggunakan .Clone() agar UI mendapatkan *salinan* gambar, bukan referensi aslinya. Jika UI mencoret-coret salinan ini, OriginalImage di dalam service tetap aman/bersih.

5. Manipulasi Kanal Warna (Channel)

C#

```
public Bitmap GetChannel(int channel)
{
    if (!IsImageLoaded) return null;
    return ImageDataProcessor.CreateBitmapFrom3DArray(_imageData3D, channel);
}
```

```
public Bitmap ApplyGrayscale()
{
    return GetChannel(3);
}
```

- **GetChannel(int channel)**: Mengambil satu lapisan warna spesifik (misal: Merah saja, atau Hijau saja). Ia menggunakan data dari _imageData3D untuk menyusun kembali menjadi Bitmap baru.
- **ApplyGrayscale()**: Secara spesifik memanggil GetChannel dengan parameter 3.
 - *Catatan:* Dalam banyak implementasi kustom, indeks 0, 1, 2 biasanya R, G, B, dan indeks 3 sering kali dialokasikan untuk hasil rata-rata (Grayscale) atau Alpha.

6. Operasi Pemrosesan Gambar (Filters)

Bagian ini memanggil logika statis dari kelas lain. Polanya serupa: Cek apakah gambar ada, lalu panggil fungsi logika.

C#

```
public Bitmap ApplyNegation(Bitmap currentImage)
{
    if (!IsImageLoaded) return null;
    return ImageNegation.Apply(currentImage);
}
```

- **ApplyNegation:** Membuat efek negatif film. Mengambil currentImage sebagai input (bukan OriginalImage), memungkinkan operasi bertumpuk (negasi dari hasil edit sebelumnya).

C#

```
public Bitmap ApplyBinarization(int step)
{
    if (!IsImageLoaded) return null;
    return BlackWhite.ApplyBinarization(OriginalImage, step);
}
```

- **ApplyBinarization:** Mengubah gambar menjadi hitam putih murni berdasarkan nilai ambang batas (step). Menggunakan OriginalImage sebagai sumber.

C#

```
public Bitmap ApplyBrightness(int adjustment)
{

```

```

        if (!IsImageLoaded) return null;
        return Brightness.AdjustBrightness(OriginalImage, adjustment);
    }

```

- **ApplyBrightness:** Mencerahkan atau menggelapkan gambar. Nilai adjustment positif untuk lebih terang, negatif untuk lebih gelap.

7. Fitur Seleksi Warna

C#

```

public Bitmap ApplyColorSelection(Point imagePoint)
{
    if (!IsImageLoaded) return null;
    Color clickedColor = OriginalImage.GetPixel(imagePoint.X, imagePoint.Y);
    return SelectionColor.ApplySelection(OriginalImage, clickedColor);
}

```

- **Color clickedColor = ...:** Mengambil warna spesifik pada koordinat (x, y) di mana pengguna mengklik (imagePoint).
- **SelectionColor.ApplySelection(...):** Memproses gambar untuk menonjolkan warna yang diklik tadi, atau mungkin mengubah area dengan warna tersebut.

8. Histogram

C#

```

public Bitmap GetHistogram(int channel, int width, int height)
{
    if (!IsImageLoaded) return null;
    int[] hist = HistogramCalculator.Calculate(OriginalImage, channel);
    return HistogramDrawer.Draw(hist, channel, width, height);
}

```

- Metode ini melakukan dua hal:

1. **HistogramCalculator.Calculate**: Menghitung statistik frekuensi kemunculan warna (matematika). Outputnya adalah array integer (int[]).
2. **HistogramDrawer.Draw**: Mengambil data statistik tadi dan menggambarinya menjadi sebuah gambar grafik (Bitmap) sesuai ukuran width dan height yang diminta.

9. Pembersihan (Cleanup)

C#

```
public void ClearImage()
{
    OriginalImage = null;
    _imageData3D = null;
}
```

- Menghapus referensi ke gambar dan data array agar memori bisa dibersihkan oleh *Garbage Collector* ketika pengguna menutup gambar atau mereset aplikasi.

Ringkasan Alur Kerja

1. UI memanggil `InitialzeImage(bmp)` -> Service menyimpan `OriginalImage` dan membuat `_imageData3D`.
2. UI minta operasi (misal `ApplyBrightness`) -> Service memanggil kelas logika (`Brightness.AdjustBrightness`) -> Service mengembalikan `Bitmap` hasil olahan ke UI untuk ditampilkan.
3. Jika UI butuh reset -> panggil `GetRestoredImage()`.

Apakah Anda ingin saya menjelaskan lebih dalam mengenai salah satu kelas Logika yang dipanggil di sini, misalnya bagaimana **`ImageDataProcessor.LoadTo3DArray`** bekerja memecah piksel?